

COMPANION
to

ChatGPT Teaches TikZ

Chapter 11: Scopes and Transformations (I)

Contents

11.1	What This Companion Adds	2
11.2	A Transformation Vocabulary	3
11.3	The Scope Boundary	4
11.4	Translation	5
11.5	Rotation	5
11.6	Scaling	6
11.7	What Happens to Nodes	6
11.8	Combining Transformations	7
11.9	Worked Project: Reusing a Motif	7
11.10	Debugging Workshop	10
11.11	Exercises	10
11.12	Solutions	11
11.13	ChatGPT Workshop	13
11.14	Final Checklist	13

11.1 What This Companion Adds

Chapter 11 introduced local scopes and geometric transformations. This Companion develops a reusable workflow: define a small motif, place each copy inside a scope, and express the difference between copies with transformation options rather than rewritten coordinates.

Design

A scope states where an option applies. A transformation states how that local coordinate system differs from the surrounding picture.

11.2 A Transformation Vocabulary

Option or environment Purpose	
<code>scope</code>	Limits options to a group of commands.
<code>shift={x,y}</code>	Translates both coordinates.
<code>xshift=2cm</code>	Translates horizontally.
<code>yshift=1cm</code>	Translates vertically.
<code>rotate=30</code>	Rotates around the current origin.
<code>rotate around={30:(P)}</code>	Rotates around point P.
<code>scale=.7</code>	Scales uniformly.
<code>xscale, yscale</code>	Scales in only one direction.
<code>transform shape</code>	Transforms node shapes and text.

11.3 The Scope Boundary

Every option attached to a scope affects only the commands inside it.

```
\begin{tikzpicture}
  \draw (0,0) rectangle (2,1);

  \begin{scope}[shift={(3,0)}]
    \draw (0,0) rectangle (2,1);
  \end{scope}

  \draw (6,0) rectangle (8,1);
\end{tikzpicture}
```

See



Only the middle rectangle moves. The scope ends before the third drawing command.

Common Mistake

If a transformation changes later objects unexpectedly, check where its scope ends. Local transformations should not leak into unrelated parts of a picture.

11.4 Translation

Translation changes position without changing size or orientation.

```
\begin{scope}[shift={(3,1)}]
  \draw (0,0) circle (.6);
\end{scope}
```

The local origin (0,0) is displayed at the surrounding position (3,1). The drawing commands inside the scope remain unchanged.

Use `xshift` or `yshift` when only one direction changes:

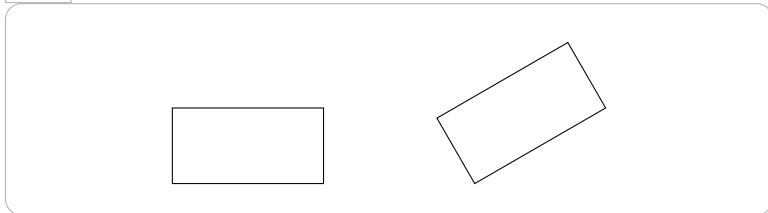
```
\begin{scope}[xshift=3cm]
  ...
\end{scope}
```

11.5 Rotation

Positive angles rotate counterclockwise; negative angles rotate clockwise.

```
\begin{tikzpicture}
  \draw (0,0) rectangle (2,1);
  \begin{scope}[shift={(4,0)},rotate=30]
    \draw (0,0) rectangle (2,1);
  \end{scope}
\end{tikzpicture}
```

See



The rectangle rotates around the local origin. To name a different center, use

```
\begin{scope}[rotate around={30:(P)}]
...
\end{scope}
```

11.6 Scaling

Uniform scaling preserves proportions.

```
\begin{scope}[scale=.6]
...
\end{scope}
```

A factor larger than 1 enlarges; a factor between 0 and 1 reduces. Nonuniform scaling changes proportions:

```
\begin{scope}[xscale=1.5,yscale=.7]
...
\end{scope}
```

Use `xscale` and `yscale` only when distortion is intentional.

11.7 What Happens to Nodes

Without `transform shape`, a node follows a transformed coordinate but its shape and text remain easy to read.

```
\begin{tikzpicture}
  \begin{scope}[rotate=30]
    \node[draw] at (0,0) {TikZ};
  \end{scope}

  \begin{scope}[
    shift={(4,0)},rotate=30,transform shape
  ]
    \node[draw] at (0,0) {TikZ};
  \end{scope}
\end{tikzpicture}
```

See

TikZ

TikZ

The second node rotates with its local coordinate system. Use this option only when the node itself should transform.

11.8 Combining Transformations

Several options may belong to one scope.

```
\begin{scope}[  
  shift={(5,1)},rotate=-25,scale=.75  
]  
  ...  
\end{scope}
```

Keep the list short enough that its purpose remains clear. If a combination is surprising, test the transformations one at a time.

11.9 Worked Project: Reusing a Motif

Define one local command for the geometry, then call it inside transformed scopes.

11.9.1 Stage 1: Define the Motif

```
\def\motif{  
  \draw[thick] (0,0) rectangle (1.6,1);  
  \draw (0,0) -- (1.6,1);  
}
```

11.9.2 Stage 2: Draw the Original and a Shifted Copy

```
\motif  
\begin{scope}[shift={(2.5,0)}]  
  \motif  
\end{scope}
```

11.9.3 Stage 3: Add a Rotated, Scaled Copy

```
\begin{scope}[shift={(5.5,0)},rotate=25,scale=.8]  
  \motif  
\end{scope}
```


11.9.4 The Complete Picture

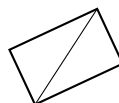
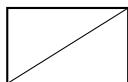
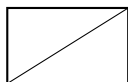
```
\begin{tikzpicture}
  \def\motif{
    \draw[thick] (0,0) rectangle (1.6,1);
    \draw (0,0) -- (1.6,1);
  }

  \motif

  \begin{scope}[shift={(2.5,0)}]
    \motif
  \end{scope}

  \begin{scope}[shift={(5.5,0)},rotate=25,scale=.8]
    \motif
  \end{scope}
\end{tikzpicture}
```

See



Design

The motif contains the geometry. Each scope contains only the difference in placement, orientation, or size.

11.10 Debugging Workshop

11.10.1 Changing Every Coordinate by Hand

If two objects have identical geometry, define that geometry once and transform a copy.

11.10.2 Forgetting a Scope Boundary

Match every `\begin{scope}` with an `\end{scope}` before adding another transformation.

11.10.3 Rotating Around the Wrong Point

Ordinary `rotate` uses the local origin. Use `rotate around` when another center is mathematically required.

11.10.4 Transforming Text Unintentionally

Remove `transform shape` when text should remain horizontal or retain its ordinary size.

11.11 Exercises

1. Draw an object and a copy shifted three units right.
2. Place three identical objects in a column with `yshift`.
3. Rotate a rectangle 30 degrees around its local origin.
4. Rotate a short segment around a named point.

5. Draw one motif at scales .5, 1, and 1.5.
6. Compare a rotated node with and without `transform shape`.
7. Combine a shift, rotation, and scale in one scope.
8. Add a fourth transformed copy to the worked project.

11.12 Solutions

11.12.1 Exercises 1–4

```
\draw (0,0) circle (.5);
\begin{scope}[shift={(3,0)}]
  \draw (0,0) circle (.5);
\end{scope}
```

```
\foreach \y in {0,2,4}
  \begin{scope}[yshift=\y cm]
    \draw (0,0) circle (.5);
  \end{scope}
```

```
\begin{scope}[rotate=30]
  \draw (0,0) rectangle (2,1);
\end{scope}
```

```
\begin{scope}[rotate around={30:(P)}]
  \draw (P) -- +(2,0);
\end{scope}
```

11.12.2 Exercises 5–7

```
\begin{scope}[scale=.5] \motif \end{scope}  
\begin{scope}[xshift=3cm] \motif \end{scope}  
\begin{scope}[xshift=6cm,scale=1.5] \motif \end{scope}
```

Exercise 6 uses the same rotated scope twice; add `transform shape` to only one version.

```
\begin{scope}[shift={(4,1)},rotate=20,scale=.8]  
  \motif  
\end{scope}
```

For Exercise 8, call `\motif` once inside one additional scope; do not copy the drawing commands.

11.13 ChatGPT Workshop

Ask ChatGPT to preserve each motif's geometry while evaluating the transformations.

1. Rewrite these coordinate-edited copies with scopes and transformations.
2. Identify exactly which commands are affected by each scope in this picture.
3. Explain the difference between **shift**, **rotate**, **scale**, and **transform shape**.
4. Diagnose why this object rotates around the wrong point.
5. Extract the repeated geometry into one local motif command and leave only transformations in the scopes.

11.14 Final Checklist

Before continuing to Chapter 12, check that you can

1. limit local options with a scope;
2. translate without rewriting coordinates;
3. rotate around the local origin or a named point;
4. distinguish uniform scaling from distortion;
5. decide whether node shapes and text should transform;
6. combine several transformations in one scope; and
7. define one motif and reuse it in transformed copies.